

■ Lab 4 — Terraform Core Behaviour & Remote State

■ Objectifs du Lab

À la fin de ce lab, vous aurez :

- Créé votre **bucket S3 personnel** avec Terraform (bootstrap)
- Configuré un **backend distant S3** avec locking natif (`use_lockfile = true`)
- Observé le comportement de Terraform face à des **modifications hors Terraform**
- Observé le comportement face à des **modifications dans le code Terraform**

■ Partie A — Bootstrap : Créer votre bucket S3 avec Terraform

■ Cette partie se fait ****une seule fois**** — le bucket sera réutilisé pour tous les labs suivants (Lab 4 à Lab 14).

Le bucket bootstrap utilise un ****state local**** car on ne peut pas utiliser S3 avant que le bucket existe.

Étape A1 — Préparer le dossier bootstrap

```
cd labs/lab4-state
mkdir bootstrap && cd bootstrap
```

Étape A2 — Créer les fichiers bootstrap

versions.tf

```
terraform {
  required_version = ">= 1.15.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "eu-west-1"
}
```

`variables.tf`

```
variable "username" {
  description = "Votre prenom - permet de nommer votre bucket de maniere unique"
  type        = string
}
```

`main.tf`

```
data "aws_caller_identity" "current" {}

resource "aws_s3_bucket" "terraform_state" {
  bucket = "tf-training-${var.username}-${data.aws_caller_identity.current.account_id}"

  tags = {
    Name      = "tf-training-${var.username}"
    ManagedBy = "Terraform"
    Username  = var.username
  }
}

resource "aws_s3_bucket_versioning" "state" {
  bucket = aws_s3_bucket.terraform_state.id

  versioning_configuration {
    status = "Enabled"
  }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "state" {
  bucket = aws_s3_bucket.terraform_state.id

  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}

resource "aws_s3_bucket_public_access_block" "state" {
  bucket = aws_s3_bucket.terraform_state.id

  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls     = true
  restrict_public_buckets = true
}
```

`outputs.tf`

```
output "bucket_name" {
  description = "Nom du bucket S3 a utiliser dans tous les labs suivants"
  value      = aws_s3_bucket.terraform_state.bucket
}
```

Étape A3 — Déployer le bucket

```
terraform init
terraform apply -var="username=<votre-prenom>"
```

Tapez `yes` pour confirmer.

Étape A4 — Sauvegarder le nom du bucket

```
# Récupérer et sauvegarder le nom du bucket
BUCKET_NAME=$(terraform output -raw bucket_name)
echo "Mon bucket S3 : $BUCKET_NAME" >> $HOME/tf-training-info.txt
cat $HOME/tf-training-info.txt
```

■ Le state du bootstrap reste **local** dans `bootstrap/terraform.tfstate` — c'est normal et voulu. Ne le supprimez pas.

■ Partie B — Configurer le Backend S3

Étape B1 — Retourner dans le dossier du lab

```
cd ..
# Vous êtes dans labs/lab4-state
```

Étape B2 — Créer les fichiers

```
lab4-state/
■■■ bootstrap/      ← déjà fait
■■■ versions.tf
■■■ variables.tf
■■■ main.tf
■■■ outputs.tf
```

`versions.tf`

```
terraform {
  required_version = ">= 1.15.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }

  backend "s3" {
    region      = "eu-west-1"
    use_lockfile = true
    encrypt     = true
  }
}

provider "aws" {
  region = "eu-west-1"
}
```

■ `use_lockfile = true` — disponible depuis Terraform 1.11. Permet le locking du state directement sur S3 ****sans DynamoDB****.

`variables.tf`

```
variable "username" {
  description = "Votre prenom - permet de differencier les ressources entre participants"
  type        = string
}
```

`main.tf`

```
resource "aws_security_group" "lab4_sg" {
  name          = "lab4-sg-${var.username}"
  description    = "Security group Lab 4 - ${var.username}"

  tags = {
    Name     = "lab4-sg-${var.username}"
    Lab      = "lab4"
    Username = var.username
  }
}

resource "aws_instance" "lab4_instance" {
  ami          = "ami-0694d931cee176e7d"
  instance_type = "t2.micro"
  vpc_security_group_ids = [aws_security_group.lab4_sg.id]

  tags = {
    Name     = "lab4-instance-${var.username}"
    Lab      = "lab4"
    Username = var.username
  }
}
```

`outputs.tf`

```
output "instance_id" {
  description = "ID de l'instance EC2"
  value       = aws_instance.lab4_instance.id
}

output "security_group_id" {
  description = "ID du Security Group"
  value       = aws_security_group.lab4_sg.id
}
```

■ Partie C — Initialiser avec le Backend S3

Étape C1 — terraform init avec backend-config

```
BUCKET_NAME=$(cat $HOME/tf-training-info.txt | awk '{print $NF}')

terraform init \
  -backend-config="bucket=${BUCKET_NAME}" \
  -backend-config="key=lab4/terraform.tfstate"
```

Résultat attendu :

```
Initializing the backend...
Successfully configured the backend "s3"!
```

Étape C2 — Déployer

```
terraform apply -var="username=<votre-prenom>"
```

Tapez **yes** pour confirmer.

Étape C3 — Vérifier que le state est dans S3

```
aws s3 ls s3://${BUCKET_NAME}/lab4/
```

Résultat attendu :

```
terraform.tfstate
```

■ Partie D — Modifier une ressource hors Terraform

```
INSTANCE_ID=$(terraform output -raw instance_id)
```

```
aws ec2 create-tags \
  --resources $INSTANCE_ID \
  --tags Key=ModifiedOutside,Value=true
```

```
terraform plan -var="username=<votre-prenom>"
```

■ *Terraform détecte que la ressource a été modifiée hors de son contrôle.*

```
terraform refresh -var="username=<votre-prenom>"
terraform state show aws_instance.lab4_instance | grep ModifiedOutside
```

■ Partie E — Modifier dans le code Terraform

Dans `main.tf`, changez `t2.micro` en `t2.small` :

```
terraform plan -var="username=<votre-prenom>"
```

■ Terraform affiche qu'il va **modifier** l'instance (~) — c'est la différence entre **desired state** et **current state**.

Remettez `t2.micro` avant de continuer :

```
terraform plan -var="username=<votre-prenom>"
```

Résultat attendu : `No changes. Infrastructure is up-to-date.`

■ Partie F — Tester le locking S3

Ouvrez **deux terminaux** dans `labs/lab4-state` :

- **Terminal 1** : `terraform apply -var="username="`
- **Terminal 2** : `terraform apply -var="username="` immédiatement après

Le Terminal 2 affiche :

```
Error: Error acquiring the state lock
```

■ Partie G — Nettoyage

```
# Détruire les ressources du lab
terraform destroy -var="username=<votre-prenom>"
```

■■ Ne détruisez **pas** le bootstrap — votre bucket S3 sera réutilisé dans tous les labs suivants !

■ Validation du Lab

#	Critère	Validé
1	Bucket S3 créé avec Terraform via le bootstrap	■
2	<code>`terraform output bucket_name`</code> retourne le nom du bucket	■
3	<code>`terraform init -backend-config`</code> réussi	■

#	Critère	Validé
4	State visible dans S3 après `terraform apply`	■
5	`terraform plan` détecte la modification hors Terraform	■
6	`terraform refresh` capture le tag ajouté manuellement	■
7	`terraform plan` détecte le changement `t2.small`	■
8	Locking S3 bloque le 2ème `apply` simultané	■
9	`terraform destroy` supprime les ressources du lab	■
10	Bucket S3 conservé pour les labs suivants	■

■ ■ Résolution des problèmes courants

Problème	Cause	Solution
`BucketAlreadyExists`	Nom de bucket déjà pris	Vérifier que le nom inclut bien votre prénom et account-id
`Backend init error`	Bucket ou key incorrects	Vérifier les valeurs `-backend-config`
`Error acquiring state lock`	Lock non libéré	`terraform force-unlock`
`NoSuchBucket`	Bucket non créé	Relancer le bootstrap
`use_lockfile not supported`	Version Terraform < 1.11	Vérifier `terraform version`

■ ****Fin du Lab 4 — Votre remote state S3 est configuré pour toute la formation !****

*Le Lab 5 introduit les ****outputs**** et les ****variables**** pour rendre votre code dynamique.*